

TRT VE TABİİ İÇERİKLERİ İÇİN İÇERİK ETKİLEŞİM BACKEND PLATFORMU

Belge Türü: Proje Tanımı ve Teknik İsterler

Belge Durumu: Planlama ve Başlangıç İskeleti

1. Giriş ve Proje Tanımı

1.1. Proje Tanımı

TRT ve tabii içeriklerini quiz ve oyunlaştırma mekanizmalarıyla etkileşimli hâle getiren, tekrar kullanılabilir bir backend platformunun geliştirilmesi hedeflenmektedir.

Platformun ilk kullanım senaryosunda yöneticiler dizi, sezon, bölüm, quiz ve soru içeriklerini yönetecektir. Kullanıcılar yayınlanmış bir bölüm quizini başlatacak, soruları belirlenen süre içinde cevaplayacak ve sunucu tarafından hesaplanan skor ile XP sonucunu görecektir. Kullanıcı ayrıca global ve içerik bazlı sıralamasını görüntüleyebilecektir.

İlk sürüm tek bir çekirdek akışa odaklanacaktır:

Yayınla -> Başlat -> Cevapla -> Tamamla -> XP Ver -> Sırala

Bu akışa doğrudan katkı sağlamayan canlı TV, eğitim modu, arkadaş sistemi, yapay zekâ ile soru üretimi ve mikroservis ayrıştırması ilk sürüm kapsamına alınmayacaktır.

1.2. Temel Amaçlar

- Backend Mühendisliği:** RESTful API, veri modelleme, transaction, concurrency, idempotency, güvenlik ve test konularını gerçek bir ürün akışı üzerinde uygulamak.
- Güvenilir Quiz Akışı:** Doğru cevap, süre, attempt durumu ve puanlama kararlarını istemciye bırakmadan sunucu tarafında güvenli biçimde yönetmek.
- Veri Bütünlüğü:** Tekrarlı cevap, tekrarlı tamamlama ve tekrarlı XP üretimi gibi hataları hem uygulama kuralları hem veritabanı constraint'leriyle engellemek.
- Modüler Tasarım:** Sistemi başlangıçta tek uygulama olarak dağıtırken identity, content, quiz, gameplay, gamification ve leaderboard sınırlarını korumak.
- Genişleyebilirlik:** Çekirdek sistem tamamlandıktan sonra yeni etkileşim türlerinin kontrollü biçimde eklenebileceği bir temel oluşturmak.

1.3. Hedef Kullanıcılar

- TRT ve tabii içeriklerini izleyen kullanıcılar
- İçerik ve quiz editörleri
- Yönetici ve operasyon ekipleri

- Gelecekte web, mobil ve Smart TV istemcileri

1.4. İlk Sürüm Kapsamı

- Dizi, sezon ve bölüm hiyerarşisinin yönetilmesi
- Quiz, quiz sürümü, soru ve cevap seçeneklerinin yönetilmesi
- Draft quiz sürümünün doğrulanarak yayınlanması
- Yayınlanmış quiz için kullanıcı attempt'i başlatılması
- Soruların süre kontrollü ve tek cevaplı biçimde yanıtlanması
- Skorum yalnız sunucu tarafından hesaplanması
- Attempt'in tek kez tamamlanması
- Tekrarlı işlem üretmeyen XP kaydı
- Global ve içerik bazlı leaderboard
- Kullanıcı, editör ve yönetici yetkilendirmesi
- OpenAPI sözleşmesi, migration ve otomatik testler

1.5. İlk Sürüm Dışındaki Konular

- Canlı TV ile eşzamanlı quiz
- İngilizce öğrenme veya eğitim modu
- Arkadaşlık, mesajlaşma ve sosyal akış
- Yapay zekâ tarafından otomatik soru üretme ve yayınlama
- Rozet sistemi
- Mikroservis mimarisi
- Kubernetes zorunluluğu
- Gelişmiş medya işleme
- Yetkisiz TRT veya tabii veri kazıma işlemleri

2. Proje Mimarisi

Sistem, frontend ve backend sorumluluklarının ayrıldığı API-first bir mimari üzerine kurulacaktır. Backend başlangıçta modüler monolith olarak geliştirilecek ve tek uygulama şeklinde dağıtılacaktır.

Modüler monolith, bütün modüllerin aynı uygulama içinde çalışmasına rağmen kod ve veri sahipliği sınırlarının korunması anlamına gelir. Mikroservis ancak bağımsız ölçekleme, ayrı ekip sahipliği, farklı servis seviyesi veya hata izolasyonu gibi ölçülebilir bir ihtiyaç oluşursa değerlendirilecektir.

2.1. Frontend: Temel Sorumluluklar

Frontend ilk sürümde backend API'lerini kullanacak kadar sade tutulacaktır. Frontend iş kurallarının güvenilir kaynağı olmayacak ve PostgreSQL'e doğrudan bağlanmayacaktır.

2.1.1. Kullanıcı Giriş ve İçerik Listeleme Ekranları

- Kullanıcı giriş işlemi için gerekli kimlik bilgileri alınmalıdır.
- Yayında bulunan içerikler, sezonlar ve bölümler listelenmelidir.
- Bir bölüm için yalnız yayınlanmış quiz bilgisi gösterilmelidir.

- Başarısız işlemlerde backend tarafından döndürülen kararlı hata koduna göre anlaşılır hata mesajı gösterilmelidir.

2.1.2. Quiz Oynama Ekranı

- Kullanıcı yayınlanmış bölüm quizini başlatabilmelidir.
- Backend tarafından gönderilen sorular ve cevap seçenekleri gösterilmelidir.
- Doğru cevap bilgisi quiz başlamadan veya cevap verilmeden istemciye gönderilmemelidir.
- Kullanıcının cevabı attempt ve soru bilgisiyle backend'e iletilmelidir.
- Frontend'deki sayaç yalnız kullanıcı deneyimi içindir; geçerli süreyi backend belirlemelidir.
- Aynı cevap isteği kullanıcı veya ağ nedeniyle tekrar gönderildiğinde backend davranışı güvenilir kalmalıdır.

2.1.3. Sonuç ve Sıralama Ekranı

- Tamamlanan attempt'ın skoru ve kazanılan XP gösterilmelidir.
- Kullanıcının global sıralaması gösterilebilmelidir.
- Kullanıcının ilgili içerik kapsamındaki sıralaması gösterilebilmelidir.
- Sonuç ekranı istemci tarafında yeniden puan hesaplamalı, backend sonucunu göstermelidir.

2.1.4. Yönetici Ekranları

- Yetkili kullanıcı içerik, sezon ve bölüm oluşturabilmelidir.
- Quiz, soru, cevap seçeneği ve doğru cevap tanımlanabilmelidir.
- Eksik veya geçersiz quiz yayınlanamamalıdır.
- Yayınlanan quiz sürümü yerinde değiştirilememeli; yeni düzenleme yeni draft sürümü oluşturmalıdır.
- Normal kullanıcı yönetici işlemlerine erişememelidir.

2.2. Backend: Teknik Mimari ve Sorumluluklar

Backend; frontend'in ihtiyaç duyduğu REST API'yi sağlayan, iş kurallarını uygulayan, yetki kontrolü yapan ve kalıcı veriyi yöneten güvenilir sistem katmanıdır.

2.2.1. Modüller

- **identity:** Kullanıcı kimliği, profil, rol ve izinleri yönetir.
- **content:** Dizi, sezon, bölüm ve yayın durumunu yönetir.
- **quiz:** Quiz, quiz sürümü, soru, seçenek ve yayınlama kurallarını yönetir.
- **gameplay:** Attempt, cevap, süre, durum geçişi ve puanlamayı yönetir.
- **gamification:** XP işlem defterini ve kullanıcı XP özetini yönetir.
- **leaderboard:** Global ve içerik bazlı sıralama sonuçlarını yönetir.
- **admin:** Yetkili yönetim use case'lerini ve audit kaydını koordine eder.
- **shared:** Yalnız gerçekten ortak olan, domain'e özel olmayan küçük teknik bileşenleri barındırır.

Bir modül başka bir modülün tablosuna doğrudan erişmemelidir. Modüller application API'leri veya ilgili aşamada tanımlanmış olaylar üzerinden haberleşmelidir.

2.2.2. Katmanlar

Her modül ihtiyaç duyduğu ölçüde aşağıdaki katmanları kullanacaktır:

- **domain:** Entity, value object, durum geçişleri ve iş kuralları
- **application:** Use case koordinasyonu ve transaction sınırı

- **ports:** Repository ve dış sistem sözleşmeleri
- **infrastructure:** PostgreSQL, framework ve ileride broker/cache adapter'ları
- **api:** Controller ile request/response modelleri

İş kuralları controller içine yazılmamalıdır. Domain modeli Spring, JPA, RabbitMQ veya Redis ayrıntılarına bağımlı olmamalıdır.

Domain entity'leri yalnız getter/setter içeren veri torbaları olmamalıdır. Örneğin attempt durum geçişleri `attempt.complete()` ve cevap kabul kuralları `attempt.submitAnswer(...)` gibi domain davranışlarıyla korunmalıdır. Değişebilir puanlama gibi kurallar saf domain policy/service nesnelerinde bulunabilir.

Modül, bir iş alanını; katman ise o modül içindeki teknik sorumluluğu ifade eder. Bütün modül ve katman klasörleri ilk günden boş olarak açılmayacak, gerçek use case ortaya çıktıkça oluşturulacaktır.

2.2.3. Modüller Arası İletişim

- Başka modüle iş yaptıran komut ve sorgular yayınlanmış application API/port üzerinden yürütülmelidir.
- Gerçekleşmiş bir iş gerçeği process içinde domain event ile bildirilebilir.
- Dış sisteme/broker'a taşınan kalıcı ve sürümlü sözleşme integration event'tir.
- Spring'in process içi event mekanizması tek başına kalıcı teslimat veya hata izolasyonu sağlamaz.
- Güvenilir dış olay teslimatında PostgreSQL Outbox, consumer tarafında Inbox/idempotency kullanılmalıdır.
- Modül sınırları gerçek kod oluştuğunda küçük ArchUnit testleriyle korunmalıdır.

2.2.4. Temel REST API Yüzeyi

- GET `/api/v1/contents`: Yayındaki içerikleri listeler.
- GET `/api/v1/episodes/{episodeId}/quizzes`: Bölümün yayınlanmış quizini listeler.
- POST `/api/v1/attempts`: Kullanıcı için yeni attempt başlatır.
- POST `/api/v1/attempts/{attemptId}/answers`: Bir soruya cevap gönderir.
- POST `/api/v1/attempts/{attemptId}/complete`: Attempt'i tamamlar ve sonucu kesinleştirir.
- GET `/api/v1/me/stats`: Kullanıcının XP ve temel istatistiklerini verir.
- GET `/api/v1/leaderboards/{scope}`: İstenen kapsamdaki sıralamayı verir.

Yönetici endpoint'leri ayrı yetki gerektirecektir. Public API sözleşmesi `/api/v1` gibi açık bir sürüm bilgisi taşıyacaktır.

2.2.5. Quiz Attempt Yaşam Döngüsü

Attempt başladığında kullanılan `quizVersionId` sabitlenmelidir. Böylece daha sonra oluşturulan yeni quiz sürümleri geçmiş sonuçları değiştiremez.

```
STARTED
|
+-- geçerli cevap --> IN_PROGRESS
|
+-- süre doldu -----> EXPIRED
|
+-- complete -----> COMPLETED
```

- Yalnız yayınlanmış quiz sürümü başlatılabilmelidir.
- Attempt kullanıcının kimliğiyle ilişkilendirilmelidir.
- Kullanıcı kimliği request payload'ından değil, doğrulanmış actor bağlamından alınmalıdır.
- Tamamlanan veya süresi dolan attempt yeni cevap kabul etmemelidir.
- Attempt yalnız bir kez tamamlanmalıdır.

- Skor, backend'in bildiği doğru cevap ve puanlama politikasına göre hesaplanmalıdır.
- Başlangıç, deadline ve cevap kabul zamanı sunucu Clock kaynağıyla belirlenmeli ve UTC saklanmalıdır.
- Complete göndermeyen attempt'in expire davranışı açıkça tanımlanmalıdır.
- Attempt kullanılan quiz ve puanlama politikası sürümünü sabitlemelidir.

2.2.6. Concurrency ve Idempotency

İki isteğin aynı zamanda gelmesi veri yarışına neden olabilir. Örneğin aynı kullanıcı aynı soruya iki paralel cevap isteği gönderebilir.

Bu risk aşağıdaki katmanlarla birlikte korunacaktır:

- Domain, attempt durumunun cevap kabul edip etmediğini kontrol eder.
- Application katmanı işlemi uygun transaction içinde yürütür.
- PostgreSQL üzerinde (attempt_id, question_id) unique constraint'i bulunur.
- Tekrar gönderilebilen komutlarda idempotency key yaklaşımı değerlendirilir.
- Aynı complete isteği ikinci skor veya XP sonucu oluşturmamalıdır.

Java içindeki synchronized gibi process içi kilitler birden fazla backend instance'ını korumadığı için veritabanı constraint ve transaction'larının yerine geçmez.

2.2.7. Güvenlik

- Authentication ile kullanıcının kimliği doğrulanmalıdır.
- Authorization ile USER, EDITOR ve ADMIN yetkileri ayrılmalıdır.
- Kullanıcı yalnız kendi attempt ve sonuçlarına erişebilmelidir.
- Yönetici endpoint'leri rol kontrolü olmadan çalışmamalıdır.
- Doğru cevap istemci DTO'suna eklenmemelidir.
- İstemciden gelen puan ve süre bilgisi güvenilir kabul edilmemelidir.
- Token, secret, doğru cevap ve kişisel veri loglanmamalıdır.
- Hata cevaplarında kararlı hata kodu ve trace ID bulunmalıdır.

2.2.8. İçerik Veri Kaynağı

Gerçek TRT veya tabii entegrasyonu için resmi ve yetkili bir API, iç servis ya da onaylı veri aktarımı gereklidir. Kamuya açık olmayan endpoint'ler otomatik kazınmamalı ve kullanım koşullarını ihlal eden veri çekme işlemi yapılmamalıdır.

Resmi entegrasyon sağlanana kadar aşağıdaki yöntemler kullanılabilir:

- Yönetici tarafından kontrollü manuel içerik girişi
- Onaylı CSV veya JSON içe aktarma
- Geliştirme ve test için sentetik örnek veri

3. Veritabanı Tasarımı

PostgreSQL; kullanıcı, içerik, quiz sürümü, attempt, cevap ve XP kayıtları için kalıcı doğru kaynak olarak önerilmektedir.

Veritabanı şeması yalnız Flyway migration dosyalarıyla değiştirilecek ve her değişiklik sürümlendirilecektir. Uygulamanın açılışta entity modelinden kontrolsüz şema üretmesi kalıcı şema yönetimi olarak kullanılmayacaktır.

3.1. Temel Veri Varlıkları

- **users:** Kullanıcı kimliği ve profil bilgileri
- **roles / user_roles:** Rol ve kullanıcı yetki eşleşmeleri
- **contents:** Dizi veya film gibi ana içerikler
- **seasons:** İçeriğe bağlı sezonlar
- **episodes:** Sezona bağlı bölümler
- **quizzes:** Bir bölüm veya içerik için quiz tanımı
- **quiz_versions:** Draft, published veya archived quiz sürümleri
- **questions:** Quiz sürümüne bağlı sorular
- **answer_options:** Soru seçenekleri ve sunucu tarafında tutulan doğruluk bilgisi
- **quiz_attempts:** Kullanıcının başlattığı quiz oturumu
- **user_answers:** Attempt içinde verilen tekil cevaplar
- **xp_transactions:** XP kazanç ve düzeltmelerinin append-only işlem defteri
- **outbox_events:** RabbitMQ aşamasında güvenilir olay yayınlama kayıtları

3.2. Temel İlişkiler

```
Content 1 --- N Season
Season 1 --- N Episode
Episode 1 --- N Quiz
Quiz 1 --- N QuizVersion
QuizVersion 1 --- N Question
Question 1 --- N AnswerOption
User 1 --- N QuizAttempt
QuizAttempt 1 --- N UserAnswer
User 1 --- N XpTransaction
```

3.3. Veri Bütünlüğü Kuralları

- Aynı içerikte sezon numarası tekrar etmemelidir.
- Aynı sezonda bölüm numarası tekrar etmemelidir.
- Quiz sürüm numarası aynı quiz içinde tekil olmalıdır.
- Yayınlanmış quiz sürümü değişmez kabul edilmelidir.
- Her soru için geçerli sayıda seçenek ve doğru cevap bulunmalıdır.
- Aynı attempt ve soru için yalnız bir user answer bulunmalıdır.
- XP kaynağı aynı kullanıcı için ikinci kez işlenmemelidir.
- Foreign key'ler sahipsiz kayıt oluşmasını engellemelidir.
- Check constraint'ler negatif puan veya geçersiz durum gibi uygun kuralları veritabanı seviyesinde de korumalıdır.

3.4. PostgreSQL, Redis ve RabbitMQ Sınırı

- **PostgreSQL:** Kalıcı iş verisinin doğru kaynağıdır.
- **XP:** Önce PostgreSQL üzerinde idempotent ledger olarak çalışacaktır.
- **Leaderboard:** Ürün kuralları ve doğru sonuç önce PostgreSQL üzerinde kanıtlanacaktır.
- **Redis:** PostgreSQL ölçümü sonrasında gerçek ihtiyaç varsa yeniden oluşturulabilir leaderboard read model için değerlendirilecektir.

- **RabbitMQ:** Çekirdek gameplay ve PostgreSQL XP güvenilir çalıştıktan sonra yan etkileri Outbox üzerinden asenkron ayırmak için değerlendirilecektir.

Redis veya RabbitMQ'nun ilk günden eklenmesi zorunlu değildir. Her teknoloji, ölçülmüş bir problem ve açık kabul kriteri olduğunda eklenmelidir.

4. Önerilen Teknolojiler

Teknoloji seçimleri kurum standardı ve backend lead görüşü alınana kadar kesin zorunluluk değil, gerekçeli öneridir.

4.1. Java 21

- Statik tip sistemi ve açık domain modelleme olanakları sunar.
- Kurumsal backend sistemlerinde olgun bir ekosisteme sahiptir.
- Thread, virtual thread, concurrency ve JVM davranışlarını öğrenme imkânı sağlar.
- Alternatif olarak ekip standardı Go ise Go; mentorluk ve kod inceleme kapasitesi nedeniyle daha doğru seçim olabilir.

4.2. Spring Boot

- REST API, dependency injection, configuration ve transaction entegrasyonunu kolaylaştırır.
- Java dilinin kendisi değildir; framework ayrıntıları domain modelinden ayrılmalıdır.
- Alternatifler arasında Quarkus, Micronaut veya ekip dili farklıysa Go/Python framework'leri bulunur.

4.3. PostgreSQL

- Transaction, foreign key, unique/check constraint, index ve locking yetenekleri nedeniyle önerilir.
- MySQL veya kurumun standart ilişkisel veritabanı makul alternatif olabilir.
- Veri bütünlüğü için yalnız uygulama kontrolüne güvenilmemelidir.

4.4. Flyway

- Veritabanı şema değişikliklerini sıralı ve tekrar uygulanabilir migration'larla sürümler.
- Liquibase veya kurumun eşdeğer migration aracı alternatiftir.
- Büyük production tablolarında index, backfill ve constraint değişiklikleri kademeli expand-contract yaklaşımıyla planlanmalıdır.
- PostgreSQL'de transaction dışında çalışması gereken DDL komutları Flyway davranışıyla birlikte ayrıca test edilmelidir.

4.5. Docker Compose

- PostgreSQL gibi yerel bağımlılıkların bütün geliştiricilerde benzer biçimde çalıştırılmasını sağlar.
- Uygulama verisi Docker volume üzerinde kalıcı tutulmalıdır.
- Production orchestration çözümü olduğu varsayılmamalıdır.

4.6. OpenAPI

- Frontend ve backend arasındaki REST sözleşmesini görünür ve test edilebilir hâle getirir.
- Endpoint, request, response ve hata davranışlarının belgelenmesini sağlar.

4.7. Testcontainers

- Integration testlerinde gerçek PostgreSQL davranışını geçici container ile doğrular.
- H2 gibi farklı davranabilen sahte bir veritabanına güvenme riskini azaltır.

- RabbitMQ ve Redis eklendiğinde ilgili gerçek container testleri de ayrıca yazılacaktır.

5. Uygulama Aşamaları, Kabul Kriterleri ve Test Yaklaşımı

5.1. Aşama 0 - Proje Temeli

İşler:

- Kurumun dil ve framework standardını doğrulamak
- Java 21 ile uyumlu ve desteklenen Spring Boot sürümünü değerlendirmek
- Maven veya Gradle seçmek
- Minimal uygulama iskeleti oluşturmak
- Yalnız PostgreSQL, Docker Compose ve Flyway altyapısını kurmak
- Health endpoint, temel hata modeli ve CI başlangıcını eklemek

Kabul Kriterleri:

- Uygulama yerel ortamda başlamalıdır.
- PostgreSQL bağlantısı kurulmalıdır.
- Migration otomatik uygulanmalıdır.
- Health endpoint başarılı cevap vermelidir.
- En az bir gerçek PostgreSQL smoke testi geçmelidir.

Test Yaklaşımı:

- Spring application context smoke testi
- Testcontainers PostgreSQL bağlantı ve migration testi
- Health ve hata zarfı için API testi

5.2. Aşama 1 - Kimlik ve Erişim Temeli

Bu aşama, içerik yönetimi ve attempt sahipliği güvenilir kullanıcı kimliğine dayansın diye öne alınmıştır.

Kabul Kriterleri:

- Korunan endpoint kimliksiz kullanılamamalıdır.
- USER, EDITOR ve ADMIN rolleri ayrılmalıdır.
- Kullanıcı kimliği request body yerine güvenilir actor bağlamından alınmalıdır.
- Kurum kimlik çözümü bilinmiyorsa kullanılan geçici yöntem açıkça belgelenmelidir.

Test Yaklaşımı:

- Authentication ve rol eşleme integration testleri
- Eksik/geçersiz kimlik ve yanlış rol API testleri
- Request body üzerinden kullanıcı taklidi yapılamadığını doğrulayan test

5.3. Aşama 2 - İçerik Kataloğu

Kabul Kriterleri:

- Dizi, sezon ve bölüm hiyerarşisi oluşturulabilmelidir.
- Aynı sezon veya bölüm numarası tekrar edememelidir.
- Yayında olmayan içerik kullanıcı API'sinde görünmemelidir.

- Normal kullanıcı yönetim endpoint'ine erişememelidir.

Test Yaklaşımı:

- Saf domain kuralları için unit test
- Foreign key ve unique constraint için PostgreSQL integration test
- Authorization, admin validation ve yayın filtresi için API test
- Modül bağımlılık yönü için ArchUnit test

5.4. Aşama 3 - Quiz Oluşturma ve Sürümleme

Kabul Kriterleri:

- Draft quiz oluşturulup doğrulandıktan sonra yayınlanabilmelidir.
- Eksik veya geçersiz quiz yayınlanamamalıdır.
- Yayınlanmış sürüm yerinde değiştirilememelidir.
- Puanlama politikasının sürümleme sınırı belirlenmelidir.

Test Yaklaşımı:

- Yayınlama kuralları için domain unit test
- Sürüm ve seçenek constraint'leri için PostgreSQL integration test
- Draft/publish/archive geçişleri için API test

5.5. Aşama 4 - Gameplay

Kabul Kriterleri:

- Kullanıcı yalnız kendi attempt'ine erişebilmelidir.
- Aynı soruya ikinci cevap kalıcı olarak engellenmelidir.
- Süresi geçen cevap sunucu saatine göre kabul edilmemelidir.
- Complete göndermeyen attempt'in expire davranışı belirli olmalıdır.
- Doğru cevap istemciye önceden gönderilmemelidir.
- Aynı complete isteği yalnız bir sonuç üretmelidir.
- Attempt quiz ve puanlama politikası sürümünü sabitlemelidir.

Test Yaklaşımı:

- Attempt durum makinesi, sahiplik, sabit Clock, süre ve skor için deterministik unit test
- Answer ve complete transaction'ları için PostgreSQL integration test
- Paralel answer/complete istekleri için concurrency ve idempotency test
- Doğru cevabın API DTO'sunda bulunmadığını doğrulayan contract test

5.6. Aşama 5 - PostgreSQL Üzerinde XP

Bu aşamada RabbitMQ kullanılmaz.

Kabul Kriterleri:

- Geçerli tamamlanmış attempt tek XP işlemi üretmelidir.
- Aynı complete veya attempt ikinci XP üretmemelidir.
- XP append-only ledger olarak saklanmalıdır.
- RabbitMQ olmadan XP özeti okunabilmelidir.

Test Yaklaşımı:

- XP politikası için saf domain unit testi
- Attempt complete ve XP transaction integration testi
- XP kaynak unique constraint ve paralel complete testi

5.7. Aşama 6 - Güvenilir Mesajlaşma ve RabbitMQ

Bu aşama çalışan gameplay ve PostgreSQL XP davranışını asenkronlaştıracaktır.

Kabul Kriterleri:

- Broker kapalıyken attempt ve Outbox kaydı kaybolmamalıdır.
- Broker geri geldiğinde Outbox olayı yayınlanmalıdır.
- Aynı olay iki kez teslim edilse de tek XP işlemi oluşmalıdır.
- Domain event ile integration event sözleşmesi ayrılmalıdır.

Test Yaklaşımı:

- XP ledger domain testleri
- Attempt complete ile Outbox kaydının aynı transaction'da olduğunu doğrulayan PostgreSQL integration testi
- Gerçek RabbitMQ ile publish, consume, retry ve duplicate testleri

5.8. Aşama 7 - PostgreSQL Üzerinde Leaderboard

Bu aşamada Redis kullanılmaz.

Kabul Kriterleri:

- En iyi/ilk/son attempt ve tekrar çözme kuralı belirlenmelidir.
- Sıralama tie-break kurallarıyla deterministik olmalıdır.
- Global ve içerik bazlı sonuç PostgreSQL'den okunabilmelidir.
- Örnek kapasite ve p95 başlangıç ölçümü kaydedilmelidir.

Test Yaklaşımı:

- Sıralama, tekrar ve tie-break için unit/property tabanlı test
- Gerçek PostgreSQL query/index integration testi
- Top N ve “benim sıram” kontrollü performans testi

5.9. Aşama 8 - Redis Leaderboard Read Model

Bu aşama PostgreSQL ölçümünde ihtiyaç görülürse veya açık öğrenme hedefiyle, gerekçesi ADR'a yazılarak uygulanacaktır.

Kabul Kriterleri:

- Sıralama tie-break kurallarıyla deterministik olmalıdır.
- Redis verisi silindiğinde leaderboard PostgreSQL'den yeniden kurulabilmelidir.
- PostgreSQL ve Redis sonuçları karşılaştırılabilir.
- Redis eklemenin ölçülen etkisi önce/sonra gösterilmelidir.

Test Yaklaşımı:

- Puan ve tie-break için unit test
- Gerçek Redis sorted set integration testi

- Redis silme, yeniden kurma ve geçici kesinti testleri

5.10. Aşama 9 - Operasyon ve Production Hazırlığı

Kabul Kriterleri:

- İstekler trace ID ile izlenebilmelidir.
- Gecikme yüzdeleri ve hata oranı ölçülebilmelidir.
- Kontrollü kesinti senaryolarında veri kaybı olmadığı gösterilmelidir.
- Backup/restore ve örnek expand-contract migration denenmelidir.
- Veri saklama/silme ve KVKK sınırları tanımlanmalıdır.

Test Yaklaşımı:

- Yük ve performans testleri
- PostgreSQL, RabbitMQ ve Redis kesinti senaryoları
- Güvenlik, rate limit ve dependency taramaları
- Backup/restore ve migration kilit riski provası

5.11. Aşama 10 - Opsiyonel Genişlemeler

Rozet, arkadaş, eğitim, canlı TV, WebSocket ve AI özellikleri çekirdek sistem ile operasyon kalitesi tamamlanmadan başlatılmayacaktır. Her biri ayrı ürün kararı, ADR, kabul kriteri ve test planı gerektirir.

6. Proje Teslimi ve Değerlendirme

- Kaynak kod sürüm kontrol sistemi içinde anlamlı commit'lerle tutulmalıdır.
- README dosyasında gerçek kurulum, çalıştırma ve test komutları bulunmalıdır.
- OpenAPI sözleşmesi üzerinden API davranışı incelenebilmelidir.
- Flyway migration'ları temiz PostgreSQL üzerinde uygulanabilmelidir.
- Kritik domain, API, migration, concurrency ve idempotency testleri otomatik çalışmalıdır.
- Kalıcı mimari kararlar ADR dosyalarıyla gerekçelendirilmelidir.
- Her aşama sonunda tamamlanan işler, test sonuçları, öğrenilen kavramlar, bilinen sorunlar ve sıradaki tek iş docs/CURRENT_STATE.md içinde güncellenmelidir.
- Demo sırasında geliştirici; request akışını, domain kurallarını, transaction sınırlarını, veritabanı constraint'lerini ve ilgili testlerin neyi kanıtladığını açıklayabilmelidir.
- Kullanılan herhangi bir kod parçasının neden yazıldığı açıklanabilmelidir.
- Bir aşama tamamlanmadan ve kullanıcı onayı alınmadan sonraki aşamaya geçilmemelidir.

7. Mevcut Durum

Proje planlama ve başlangıç ortamı hazırlığı aşamasındadır. Ürün kapsamı, modüler monolith yaklaşımı, geliştirme yol haritası ve eğitim belgeleri hazırlanmıştır.

Java 21 geliştirme ortamı kurulmuştur. Docker Desktop kurulum dosyası indirilmiş ve WSL 2 etkinleştirilmiştir. Docker Desktop'ın kurulması ve Docker Engine ile Compose komutlarının doğrulanması sıradaki tek teknik iş.

Henüz Spring Boot uygulama iskeleti, PostgreSQL container'ı, migration veya uygulama özelliği geliştirilmemiştir. Bu belge hedef sistemi ve teslim beklentilerini tanımlar; tamamlanmış özellik raporu değildir.